



人工智能数学基础

Essential Math for AI

第7章 非凸优化及启发式优化

哈尔滨工业大学计算学部人工智能专业 刘绍辉

shliu@hit.edu.cn

微信: shliu13503627854

2026年4月



提纲

- 启发式算法
- 人工神经网络
- 随机的人工神经网络
- 进化算法和遗传算法

7.1 启发式算法-例

• 大飞机的长期试飞任务规划

- 民用航空飞机从开始研发到投入运营共需要经历四个关键阶段：**研发设计阶段、飞机制造阶段、试飞阶段、认证阶段**
- **试飞阶段**是航空专家通过一系列测试验证飞机在各种飞行条件下的实际表现，测试其结构强度、操控性能、系统稳定性等，发现潜在问题并加以修正的重要阶段
- 一个**完整的试飞阶段**包含**地面滑行试验、失速试飞、飞行载荷试飞、侧风试飞**等众多专业科目，且各科目执行均需满足**严格的约束条件**以保障试飞安全与数据可靠性
- 一架新型民航飞机试飞阶段的**完整试飞任务数量规模通常上千**，一些试飞科目有特定的试飞要求和限制条件，有的要特定气象条件、跑道空域、有的要专项改装、试验设施，还有很多科目存在刚性的逻辑关系
- **高效的试飞任务调度计划表**非常困难，进而影响了整体民航飞机的试飞测试效率

7.1 启发式算法-例

- 任务规划变更



7.1 启发式算法-例

- 试飞任务规划问题是一个典型的NP-hard问题，根据其任务规划的不同能够获得不同的试飞执行时间
 - 优化目标：飞行试验周期为主要，任务试验日期评价罚函数
- 求解试飞任务规划问题的方法除了实际规划过程中的根据专家经验进行人工安排
- 采用智能优化算法求解试飞任务规划问题：遗传算法、粒子群优化、模拟退火算法等
- 极少部分采用深度强化学习算法进行探索
- 柔性作业车间调度问题
 - 建立多个工件和多个机器之间的匹配关系，每个工件存在不同的工序且工序之间存在确定的顺序。每道工序与机器之间也存在着对应关系，问题的目标是建立工件、工序和机器之间的最优匹配关系，使其在某些性能指标上到达最优
 - 优化目标：订单完成日期

7.1 启发式算法-例

- 六类约束
优先级约束、时间窗口约束、备选飞机约束、试验飞机制造约束、次要任务组合约束和型号检查核准书约束 (Type Inspection Authorization, TIA)

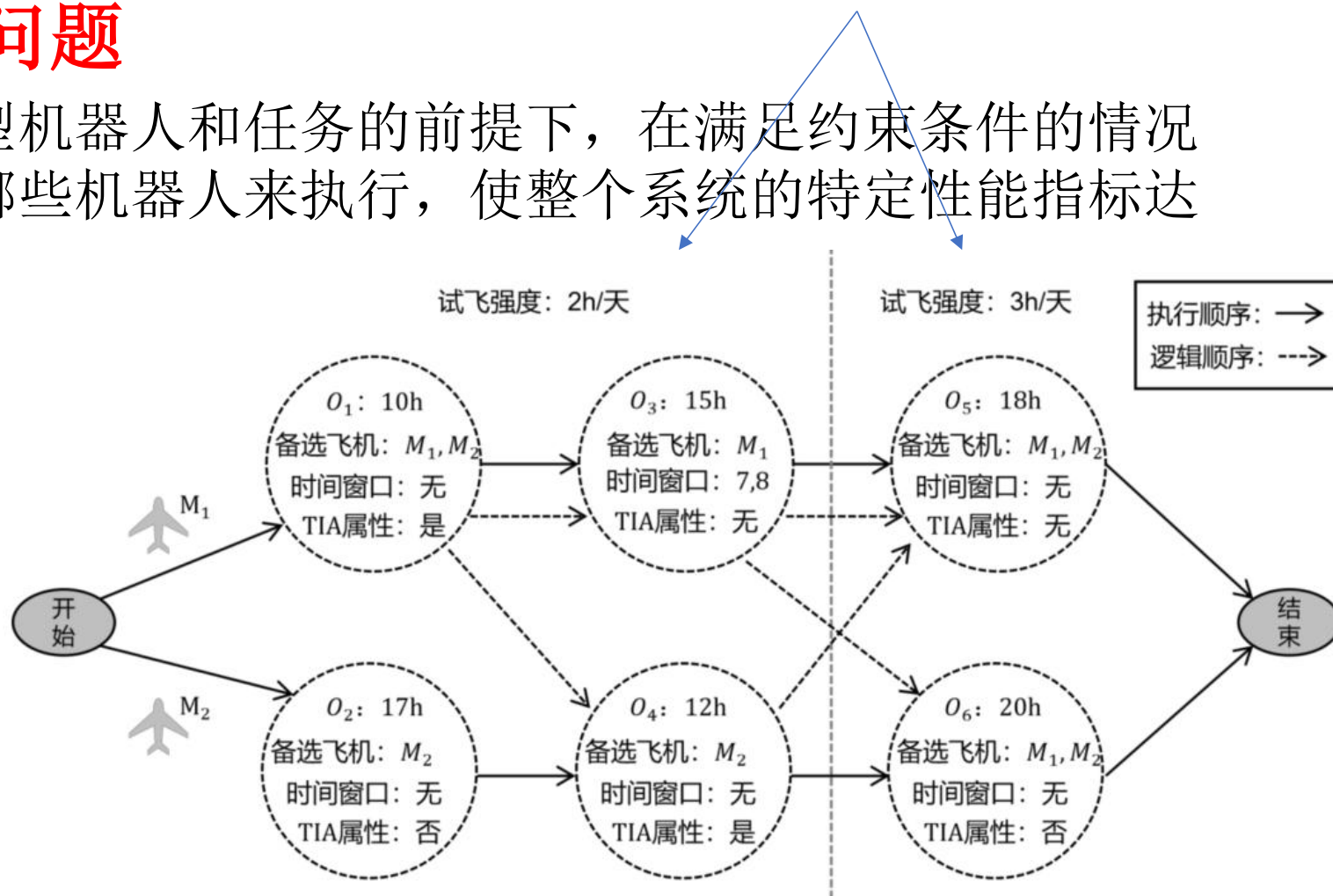
在试飞过程中，随着试飞工作的逐步推进，测试强度需依据进程动态调整，而标志这一调整的关键节点被定义为 TIA

• 多机器人任务分配问题

- 指在给定一组不同类型机器人和任务的前提下，在满足约束条件的情况下，决定哪些任务由哪些机器人来执行，使整个系统的特定性能指标达到最优

• 试飞任务建模

已知： 试验飞机编号及其投入启用时间、试飞科目编号及试飞科目之间的强逻辑关系、试飞科目与试验飞机之间的匹配关系、试飞科目的固有特性（比如科目执行时长、特定执行时间窗口、是否具备次要任务等等）、飞机的试飞强度等



7.1 启发式算法-例

- m 架试验飞机($k = 1,2, \dots, m$)、 n 个待分配的试飞任务($i = 1,2, \dots, n$)，任务与任务之间存在着优先级约束、次要任务组合约束等逻辑关系，任务与试验飞机之间存在着备选飞机约束等匹配关系以及部分任务存在着时间窗口约束等自身特性。现需要在满足所有硬约束前提下，将所有试飞任务合理分配到试验飞机上，以达到整体试飞周期时长最小的效果

任务编号	持续时间(h)	前置任务	时间窗口	可选试验飞机	是否为TIA前任务
O ₁	5	无	无	M ₁ 、M ₂	否
O ₂	7.5	O ₁	无	M ₂	否
O ₃	6	O ₁	无	M ₁	否
O ₄	4	无	1、2	M ₁	否
O ₅	10	O ₄	无	M ₁ 、M ₂	否
O ₆	4.5	O ₅	无	M ₁ 、M ₂	否
O ₇	5	无	7、8	M ₂	否

表格给出了一个简单的试飞任务测试案例，本案例中包含 11 个试飞任务（O₁-O₁₁）和两架试验飞机（M₁， M₂）

O ₈	8	无	无	M ₁ 、M ₂	否
O ₉	6	O ₃	无	M ₁ 、M ₂	是
O ₁₀	3	O ₉	无	M ₁	是
O ₁₁	5	O ₁₀	无	M ₁	是

7.1 启发式算法-例

i : 试飞任务编号索引。 k : 试验飞机编号索引。

h : 试飞任务在试验飞机上执行的序列索引, 当 $h = e$ 时, 表示飞机执行的最后一个任务。

m : 试验飞机总架数。 n : 试飞任务总数量。

$T_{i,k}$: 表示第 i 号任务在第 k 号飞机上的完成时刻。

$t_{i,k}$: 表示第 i 号任务在第 k 号飞机上的开始时刻。

P_i : 表示第 i 号任务的前置任务集合。

π_i : 表示第 i 号任务的任务执行时长。

C_i : 表示第 i 号任务的可选飞机集合。

T_{ai} 、 T_{bi} : 分别表示第 i 号任务时间窗口的下界和上界。

T_n : 表示时间窗口集合。

μ : 表示试飞测试强度。

μ_0 、 μ_1 : 常数, μ_0 表示 TIA 前设定的试飞测试强度值, μ_1 表示 TIA 后设定的试飞测试强度。

S_{TIA} : 表示试飞过程的 TIA 状态, 如果 TIA 前需完成的任务未全部完成则为0, 否则为 1。

$M_{k,i}$: 在发生动态事件时, 表示原调度技术中试验飞机 k 是否执行了任务 i , 如果执行了则 $M_{k,i}$ 为 1, 否则为 0。

$M_{kr,i}$: 在发生动态事件时, 表示新调度计划中试验飞机 k 是否执行了任务 i 如果执行了则 $M_{kr,i}$ 为 1, 否则为 0。

$\delta(M_{kr,i}, M_{k,i})$: 在发生动态事件时, 当 $\delta(M_{kr,i}, M_{k,i})$ 为 1 时, 表示任务 i 发生了飞机-任务偏移, 否则为 0。

S_i 、 C_i : 在发生动态事件时, 分别表示原调度计划中任务 i 的执行开始时刻和执行完成时刻。

S_{ir} 、 C_{ir} : 在发生动态事件时, 分别表示新调度计划中任务 i 的执行开始时刻和执行完成时刻。

$X_{i,k,h}$: 0-1 决策变量, 如果第 i 号任务被分配给第 k 架飞机, 且为该飞机执行的第 h 项任务, 则为 1, 否则为 0。

y_i : 0-1 决策变量, 如果第 i 号任务存在时间窗口则为 1, 否则为 0。

ϕ_k : 表示第 k 架飞机的试飞完成天数。

TAD: 发生动态事件时, 表示任务-飞机偏离量。

TOD: 发生动态事件时, 表示任务顺序偏离量。

目标函数: $Min\{Max\{\phi\}\}$, $\phi = \{\phi_1, \phi_2, \dots, \phi_m\}$

7.1 启发式算法-例

- 静态试飞任务规划

- $Min\{Max\{\phi\}\}$, $\phi = \{\phi_1, \phi_2, \dots, \phi_m\}$

- 动态试飞任务规划：初始调度计划发生变化，避免资源浪费

- 用偏移量来描述动态事件发生后的调度计划与原计划之间的偏差，包含任务-飞机偏移量**TAD**与任务顺序偏移量**TOD**两个指标，偏移量越小时说明动态事件发生后修正的调度计划更优

$$TAD = \sum_{i=1}^n \delta(M_{k,i}^r, M_{k,i}),$$
$$\delta(M_{k,i}^r, M_{k,i}) = \begin{cases} 1, & M_{k,i}^r \neq M_{k,i} \\ 0, & M_{k,i}^r = M_{k,i} \end{cases}$$

$$TOD = \sum_{i=1}^n \sum_{j=i+1}^n TOD_{i,j},$$
$$TOD_{i,j} = \begin{cases} 1, & (C_i \leq S_j \& \& C_j^r \leq S_i^r) \vee (C_j \leq S_i \& \& C_i^r \leq S_j^r) \\ 0, & M_{k,i}^r = M_{k,i} \end{cases}$$

7.1 启发式算法-例

• 试飞任务规划的约束

$$\left\{ \begin{array}{l}
 \phi_k = x_{i,k,e} T_{i,k} \rightarrow \text{表示每架飞机的试飞时间为该飞机上最后一项任务的完成时间} \\
 x_{i,k,h} t_{i,k} - \max \{T_{j,l} x_{j,l,g}\} \geq 0 (i = 1, 2, \dots, n; j \in P) \rightarrow \text{确保每项任务仅在其所有前置任务完成后才开始执行, 以此维持任务的优先级约束} \\
 \sum_k x_{i,k,h} = 1, i = 1, 2, \dots, n, k \in C \\
 T_{i,k} - t_{i,k} = \frac{\pi_i x_{i,k,h}}{\mu}, i = 1, 2, \dots, n \rightarrow \begin{array}{l} \text{保证每项任务恰好被分配给一架符合条件的飞机} \\ \text{确保任务一旦开始便持续执行, 中途不中断} \end{array} \\
 T_{ai} \leq x_{i,k,h} t_{i,k} \leq T_{bi}, T_{ai} \in T_n, T_{bi} \in T_n, y_i = 1 \rightarrow \text{有指定时间窗口的任务施加时间窗口约束, 确保其开始时间和完成时间均处于允许的时间范围内} \\
 T_{ai} \leq x_{i,k,h} T_{i,k} \leq T_{bi}, T_{ai} \in T_n, T_{bi} \in T_n, y_i = 1 \\
 \mu = (1 - T_{TIA})\mu_0 + S_{TIA} * u_1 \rightarrow \text{定义了试飞的飞行测试强度} \\
 x_{i,k,h} \in \{0, 1\} \\
 y_i \in \{0, 1\} \rightarrow \text{确保任务被分配给符合条件的飞机, 以及特定任务遵守时间窗口约束}
 \end{array} \right.$$

$x_{i,k,h}$: 0-1 决策变量, 如果第*i*号任务被分配给第*k*架飞机, 且为该飞机执行的第*h*项任务, 则为1, 否则为0。

y_i : 0-1 决策变量, 如果第*i*号任务存在时间窗口则为1, 否则为0

7.1 启发式算法

- 最优化算法
 - 求解该问题的每个实例的解
 - 尤其是线性规划，凸规划问题
 - 但需要在精度和速度之间做均衡!
- 启发式算法 (heuristic)
 - 一种技术，使得在可接受的计算代价内寻找到最好的解，但不一定能保证所得解的可行性和最优性
 - 甚至在多数情况下，无法阐述所得解同最优解的近似程度
 - 例如TSP问题（旅行商问题）

城市数	24	25	26	27	28	29	30	31
时间	1s	24s	10m	4.3h	4.9d	136.5d	10.8y	325y

7.1 启发式算法

- 启发式算法的特点
 - 不考虑算法所得解与最优解的偏离程度
 - 若采用分析最坏情况下的误差界限来评价启发式算法
- 近似算法
 - 设 A 是一个问题。记问题 A 的任何一个实例 I 的最优解和启发式算法 H 解的目标值分别为 $z_{OPT}(I)$ 和 $z_H(I)$ 。于是对于某个 $\epsilon \geq 0$,称 H 是 A 的 ϵ 近似算法 (ϵ -*approximation algorithm*) 当且仅当
$$|z_H(I) - z_{OPT}(I)| \leq \epsilon |z_{OPT}(I)|, \forall I \in A$$
 - 启发式算法与近似算法的区别
 - 启发式算法集合包含近似算法集合
 - 近似算法强调给出算法最坏情况的误差界限, 启发式算法不考虑
 - 最坏情况误差界估计需要较好的数学和技巧, 即使这样也很难, 只能通过启发式算法解决

7.1 启发式算法

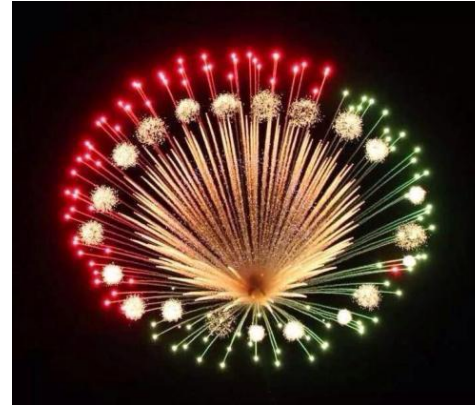
- 启发式算法如背包问题的贪婪算法 (greedy)
- 近年来得到快速发展
 - 数学模型是实际问题的简化
 - 忽略因素、数据不精确、参数估计不准确，从而使得最优化算法比启发式算法的解更不准确
 - 有些难的组合优化问题可能还没有找到最优算法，即使存在，由算法复杂性理论，其计算时间也无法接受，超出实际计算能力
 - 一些启发式算法可以用在最优算法中，如分支定界算法中可以用启发式算法估界
 - 简单易行；直观易被接受
 - 速度快，适合实时应用
 - 多数情况下，程序简单，易于修改
- 缺点
 - 不能保证求得最优解
 - 表现不稳定
 - 算法好坏依赖于实际问题，不同算法之间难以比较

概率算法：随机优化算法

墨菲定律和幸存者偏差

Murphy's Law

7.1 启发式算法



- 现代优化算法：大都具有一定的随机性
 - 禁忌搜索(tabu search)
 - 模拟退火 (Simulated annealing)
 - 遗传算法 (genetic algorithms) :在图像处理、信号处理、模式识别等领域有着广泛的应用
 - 人工神经网络 (neural networks)
 - 粒子群优化算法 (particle swarm optimization) 一种模仿鸟群、鱼群觅食行为发展起来的一种进化算法
 - 差分进化算法 (Differential Evolution) :群体进化,模拟了群体中的个体的合作与竞争的过程
 - 人工蜂群算法 (Artificial Bee Colony Algorithm, ABC)
 - 蚁群优化算法 (Ant Colony Optimization)
 - 人工鱼群优化算法 (Artificial Fish Swarm Algorithm)
 - 菌群优化算法Bacterial Foraging Optimization
 - 杜鹃搜索算法 (Cuckoo search, CS) 模仿杜鹃鸟寻窝产卵活动的群集智能优化算法
 - 群搜索算法 (Group Search Optimizer) 是一种基于发现者, 跟随者, 游荡者模型而产生的算法
 - 灰狼算法 (Grey Wolf Optimizer)
 - 萤火虫算法(Firefly Algorithm)是一种模仿萤火虫之间信息交流, 相互吸引集合, 警戒危险
 - 烟花优化算法 (Fireworks Algorithm)
 - 鲸鱼算法 (Whale Optimization Algorithm)
 - 水波算法 (Water wave optimization) 是根据水波理论提出的优化算法。什么是水波理论? 简单来说就是水波的宽度越小, 其频率越高, 频率与水波宽度的平方根成反比

7.1 启发式算法

• 基本起源

算法

1遗传算法

2粒子群优化算法

3差分进化算法

4人工蜂群算法

5蚁群算法

6人工鱼群算法

7杜鹃搜索算法

8萤火虫算法

9灰狼算法

10鲸鱼算法

11群搜索算法

12混合蛙跳算法

13烟花算法

14菌群优化算法

由来

进化论

鸟群觅食

进化论

蜜蜂觅食

蚂蚁觅食

鱼群觅食

杜鹃产卵

萤火虫求偶

狼群觅食

鲸鱼觅食

生产消费跟随模型

青蛙觅食

烟花

菌群生长

类别

人类理论

生物生存

人类理论

生物生存

生物生存

生物生存

生物生存

生物生存

生物生存

生物生存

人类理论

生物生存

物理现象

生物生存

7.1 启发式算法

• 更新策略

跟随：往最优解靠近，
依赖群体合作，**群智算**
法

不跟随：随机，更多样，
但有适合度函数判断好坏，
进化算法

算法

1遗传算法

2粒子群优化算法

3差分进化算法

4人工蜂群算法

5蚁群算法

6人工鱼群算法

7杜鹃搜索算法

8萤火虫算法

9灰狼算法

10 鲸鱼算法

11群搜索算法

12混合蛙跳算法

13烟花算法

14菌群优化算法

类别

不跟随最优解

跟随最优解

不跟随最优解

跟随最优解

跟隨最优解

跟随最优解

跟随最优解

跟随最优解

跟随最优解

跟随最优解

跟随最优解

跟随最优解

跟随最优解

跟随最优解

7.1 启发式算法

- 烟花算法 (Fireworks Algorithm: FWA)
 - 每一个烟花的位置都代表一个可行解
 - 爆炸产生：正常的火星与特别的火星。
 - 每个火星都会爆炸产生数个正常火星，某些火星有一定的概率产生一个特别的火星。正常的火星根据当前火星的振幅随机均匀分布在该火星的周围，而特别的火星将在当前火星附近以正态分布方式产生。每次迭代产生的火星数量多于每一代应有的火星数，算法将参照火星位置的优劣，随机留下指定数量的火星，以保持火星数目的稳定。

7.1 启发式算法

- 烟花算法

- 火星：位置 X 以及振幅 A ,

- 振幅与所有火星的亮度值有关，亮度越高的火星的振幅越小，反之振幅越大。（这里亮度表示火星的适应度值，越亮表示这个位置的氧气浓度越高，适应度越好）

适应度值 f 越小越优的情况:

$$A_i = A_{max} \frac{f(X_i) - f_{min} + \xi}{\sum_{k=1}^N (f(X_k) - f_{min}) + \xi}, f_{min} = f_{best}$$

适应度值 f 越大越优的情况:

$$A_i = A_{max} \frac{f_{max} - f(X_i) + \xi}{\sum_{k=1}^N (f_{max} - f(X_k)) + \xi}, f_{max} = f_{best}$$

产生正常火星的数量:

$$S_i = S_{max} \frac{f(X_i) - f_{min} + \xi}{\sum_{k=1}^N (f(X_k) - f_{min}) + \xi}, f_{max} = f_{best}$$

$$S_i = \begin{cases} \text{round}(a * S_{all}), S_i < a * S_{all} \\ \text{round}(b * S_{all}), S_i > b * S_{all}, a < b < 1 \\ \text{round}(S_i), a * S_{all} \leq S_i \leq b * S_{all}, a < b < 1 \end{cases}$$

$$S_i = S_{max} \frac{f_{max} - f(X_i) + \xi}{\sum_{k=1}^N (f_{max} - f(X_k)) + \xi}, f_{min} = f_{best}$$

7.1 启发式算法

- 产生正常火星 $X_{i,j}^{k+1} = \begin{cases} A_i * rand(-1, 1) + X_{i,j}^k, j \in \{d_1, d_2, \dots, d_z\}, 1 \leq z \leq d \\ X_{i,j}^k, j \notin \{d_1, d_2, \dots, d_z\} \end{cases}$
 - z 为随机正整数

$$X_{i,j}^{k+1} = \begin{cases} randGauss(1, 1) * X_{i,j}^k, j \in \{d_1, d_2, \dots, d_z\}, 1 \leq z \leq d \\ X_{i,j}^k, j \notin \{d_1, d_2, \dots, d_z\} \end{cases}$$

- 产生特别火星

- 火星保留到下一代

- N 个火星：产生 S_{all} 个正常火星至下一代

- 每次从中选择最优的火星

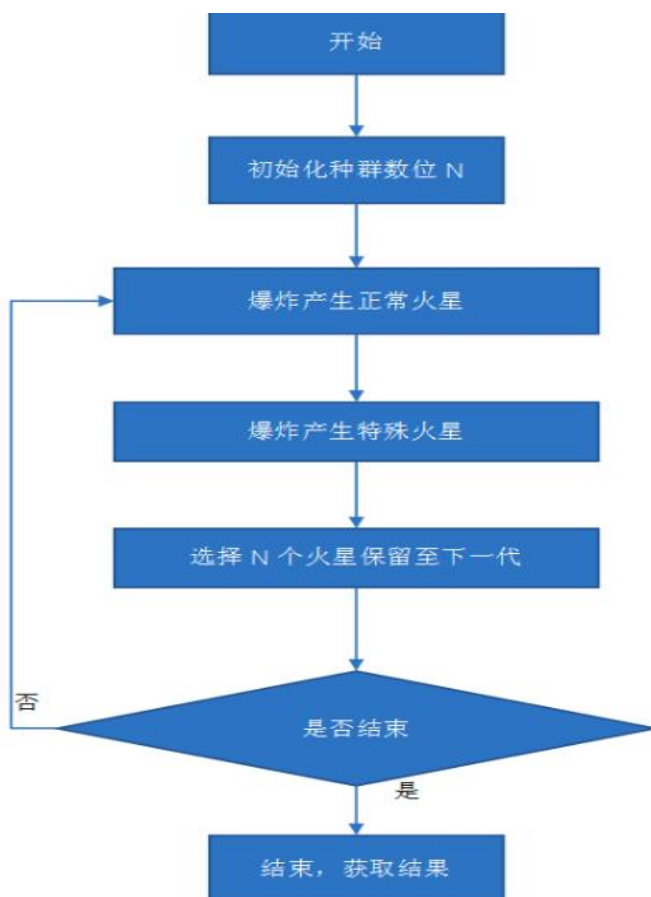
$$p(X_i) = \frac{R(X_i)}{\sum_{k=1}^{S_{all}+m+N} R(X_k)} \quad \text{代中只能从这 } N + S_{all} + m \text{ 个火星中选择 } N \text{ 个火星保留}$$

$$R(X_i) = \sum_j R(X_j)$$

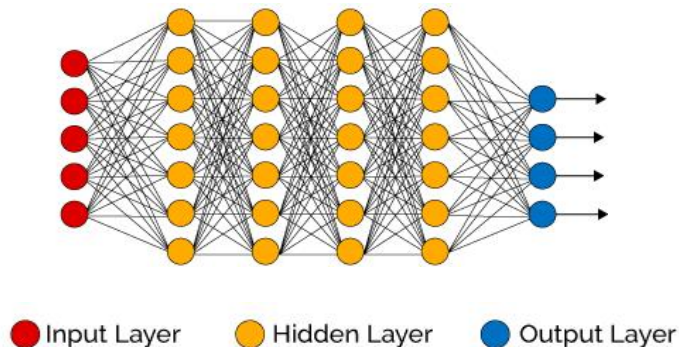
- $R(X)$ 表示该火星距其他所有火星的距离之和，即距其它火星越远的火星，被选择保留至下一代的概率较大

7.1 启发式算法

- 烟花算法



7.2 人工神经网络



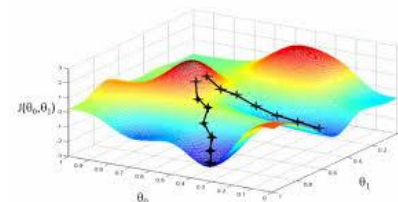
θ : 神经网络参数
 $(x_1, y_1), (x_2, y_2), \dots$ iid (样本, 标签) (*training data*)

$\ell(\theta, x, y)$ 损失函数(网络输出与真实输出之间的差别)
可以是各种类似性的损失函数, 如 l_2 , cross-entropy...

目标函数: $\operatorname{argmin}_{\theta} E_i[\ell(\theta, x_i, y_i)]$

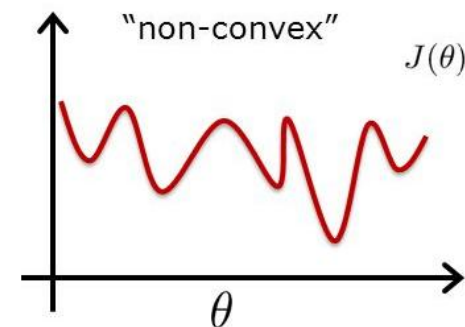
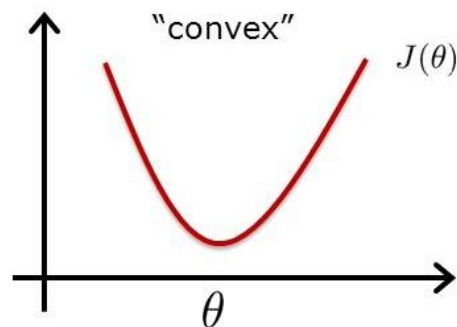
梯度下降(GD): $\theta^{(t+1)} \leftarrow \theta^{(t)} - \eta \nabla_{\theta} (E_i[\ell(\theta^{(t)}, x_i, y_i)])$

随机梯度下降(StochasticGD): 通过少量训练数据来估计 ∇



7.2 人工神经网络与优化

- 深度学习中的大多数优化问题都是非凸的
 - 机器学习也类似
 - 甚至有可能是NP-Hard的



- 可控梯度下降
 - $\nabla \neq 0 \rightarrow \exists$ 下降方向, $\theta_{t+1} = \theta_t - \eta \nabla f(\theta_t)$
 - 如果Hessian矩阵 (∇^2) 大, 允许梯度 ∇ 有较大的波动!
 - 为确保下降, 根据平滑性选取小步

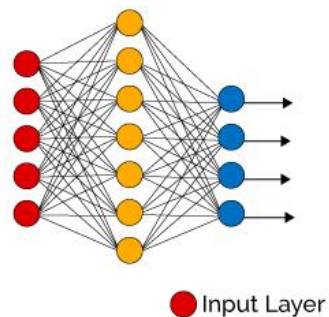
$$\nabla^2 f(\theta) \leq \beta I$$

7.2 人工神经网络与优化

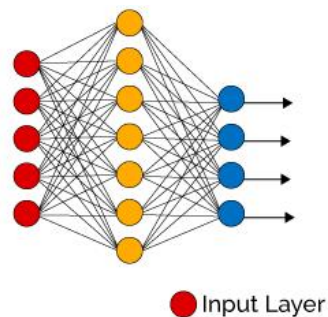
- 平滑性: $-\beta I \preceq \nabla^2 f(\theta) \preceq \beta I$
- 梯度下降算法
 - $\theta_{t+1} = \theta_t - \eta \nabla f(\theta_t)$
 - 若 $\eta = 1/2\beta$, 则在与 β/ϵ^2 成正比的步数中可以达到 $|\nabla f| < \epsilon$
 - $$\begin{aligned} f(\theta_t) - f(\theta_{t+1}) &\geq -\nabla f(\theta_t)(\theta_{t+1} - \theta_t) - \frac{1}{2}\beta|\theta_t - \theta_{t+1}|^2 \\ &= \eta|\nabla_t|^2 - \frac{1}{2}\beta\eta^2|\nabla_t|^2 = \frac{3}{8\beta}|\nabla_t|^2 \end{aligned}$$
 - 更新导致函数值减少 $3\epsilon^2/8\beta$
- 深度学习中也可以使用二阶
 - 但二阶渐近快于一阶
 - 但到目前为止, 二阶看起来并没有获得更好质量的解!

7.2 人工神经网络与优化

- 思考



给隐层单元数目为 n 的深度为2的网络
随机输入产生带标签的数据



用这些样本很难训练出一个具有同样
隐含层节点数目的新网络

迄今没有理论证明，但训练一个具有更多隐含层或更多节点的新网络则更容易

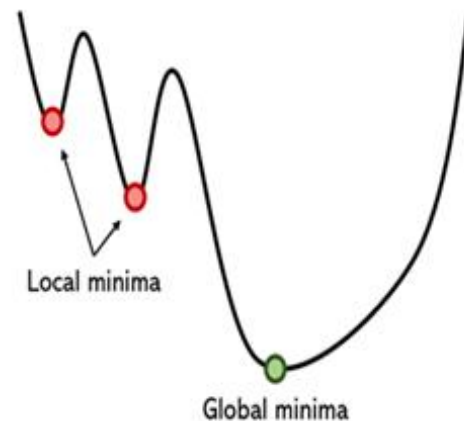
7.3 随机神经网络

- BP网络

- 可能陷入局部最优？
- 梯度下降：算法赋予网络的是只会“下山”而不会“爬山”的能力

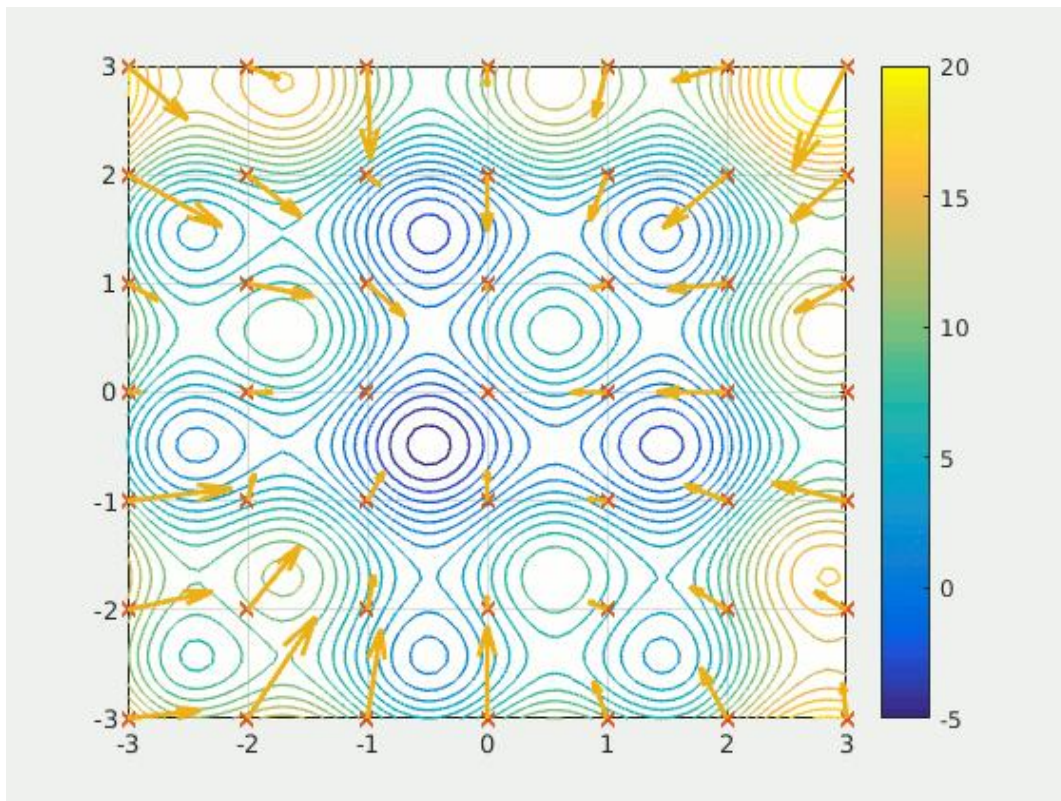
- 由于优化器深受局部极小值驱使，即便它极力摆脱后者，也要用很长时间。因收敛速度慢，梯度下降法通常十分冗长，即使是适应像批量梯度下降这样的大数据集之后也是如此。
- 学习率决定优化器的可靠程度和风险度。学习率设置过高可能会导致优化器忽略全局最小值，而过低则会导致运行时崩溃。解决此问题需要设置随着衰减而变化的学习率，但是在决定学习率的许多其他变量中选择衰减率很困难
- 梯度下降对优化器的初始化尤为敏感。例如，优化器在第二个局部最小值而非第一个局部最小值附近进行初始化，则优化器的性能可能会更优，但这都是随机决定的。
- 梯度下降需要梯度，这意味着除了无法处理不可微函数之外，还容易出现诸如梯度消失或梯度爆炸等梯度问题。

损失函数



7.3 随机神经网络

- PSO粒子群优化算法找极值点



7.3随机神经网络

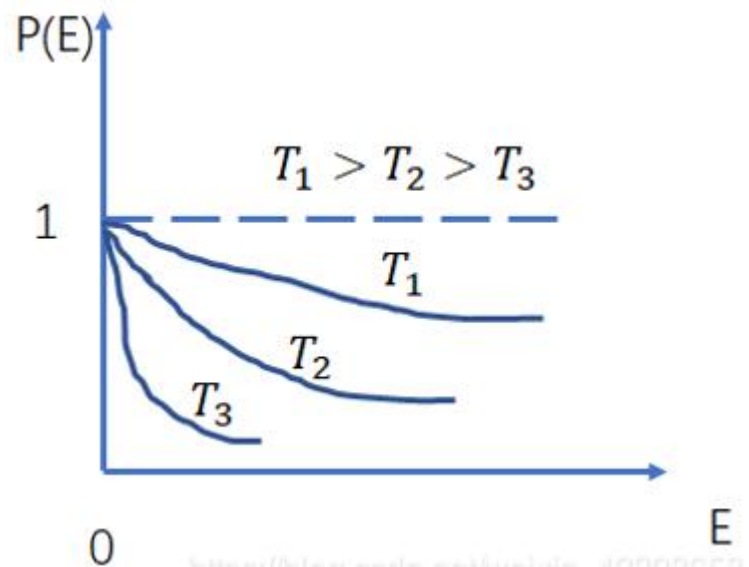
- 有没有方法改进这种情况？
 - (1)在学习阶段，随机网络不像其他网络那样基于某种确定性算法调整权值，而是按某种概率分布进行修改。
 - (2)在运行阶段，随机网络不是按某种确定性的网络方程进行状态演变，而是按某种概率分布决定其状态的转移。神经元的净输入不能决定其状态取1还是取0，但能决定其状态取1还是取0的概率
- 避免陷入局部极小点
 - 模拟退火算法是随机网络中解决能量局部极小问题的一个有效方法
 - 模拟金属退火过程，先将物体加热至高温，使其原子处于高速运动状态，此时物体具有较高的内能；然后，缓慢降温，随着温度的下降，原子运动速度减慢，内能下降；最后，整个物体达到内能最低的状态。这个状态就是最优解

7.3随机神经网络

- 在随机网络学习过程
 - 网络权值作随机变化,
 - 计算变化后的网络能量函数
 - 网络权值的修改准则
 - 若权值变化后能量变小, 则接受这种变化;
 - 否则按预先选定的概率分布接受权值的这种变化
 - 从而赋予网络一定的“爬山”能力
 - Metropol等提出的模拟退火算法
- 设 X 代表某一物质体系的微观状态 (一组状态变量, 如粒子的速度和位置等), $E(X)$ 表示该物质在某微观状态下的内能, 对于给定温度 T , 如果体系处于热平衡状态, 则在降温退火过程中, 其处于某能量状态的概率与温度的关系遵循Boltzmann分布规律。

7.3随机神经网络

- 分布函数为
- $P(E) = e^{-\frac{E(X)}{KT}}$
- 其中K为玻尔兹曼常数
- 温度一定，能量越高，概率越低
- 物质趋向于向能量低的地方演变
- 由此可见，温度参数T越高，状态越容易变化“为了使物质体系最终收敛到低温下的平衡态，应在退火开始时设置较高的温度，然后逐渐降温，最后物质体系将以相当高的概率收敛到最低能量状态。用随机神经网络解决优化问题时，通过数学算法模拟了以上退火过程



7.3 随机神经网络

- 模拟退火

- 固体加热后再慢慢冷却，固体在压力下加热，其内部能量增加且分子随机运动。随后，固体慢慢冷却，热能一般会慢慢减少，但有时也以Boltzmann概率随机增加，即在温度 t ，能量增加幅度 ΔE 的概率密度为 $e^{-\frac{\Delta E}{kt}}$ ，其中 k 是Boltzmann常数。如果冷却相当慢且降温足够大，则最终状态是无压力下的，且所有分子都按最小势能形式排列
- 如何与优化问题对应？
 - 在 $x \in \mathcal{X}$, $\min f(x)$ 的优化问题，用上述物理冷却过程来求解一个组合优化问题。对模拟退火算法， x 对应材料的状态， $f(x)$ 对应其能量水平，最优解 x^* 对应具有最小能量的材料状态。
 - 当前状态间的随机转换，即由 $x^{(k)} \rightarrow x^{(k+1)}$ 的移动由上述的Boltzmann分布决定，此分布依赖温度参数 t
 - 温度高，更可能接受上坡移动，即向更高能的状态移动，则阻止算法收敛到已经找到的第一个局部最小值；随着温度降低，逐渐收敛到整体最小值

7.3随机神经网络

- 模拟退火

- $k = 0$ 的初值为 $\mathbf{x}^{(0)}$,温度为 t_0 , k 表示迭代变量, 此算法在几个阶段内运行, 阶段标号用 $j = 0, 1, 2, \dots$ 表示, 每个阶段含有多步迭代, 第 j 个阶段的长度为 m_j , 每次迭代如下:

- 1.在 $\mathbf{x}^{(k)}$ 的邻域内 $N(\mathbf{x}^{(k)})$, 根据提案密度 $g^{(k)}(\cdot | \mathbf{x}^{(k)})$ 选取候选解 $\mathbf{x}^*$;
- 2.随机决定是否采用 $\mathbf{x}^*$ 作为下一个候选解或还是仍用当前解, 特别地, 以概率:

$$\min \left(1, e^{\left\{ \frac{[f(\mathbf{x}^{(k)}) - f(\mathbf{x}^*)]}{t_j} \right\}} \right) \text{取} \mathbf{x}^{(k+1)} = \mathbf{x}^*, \text{否则令} \mathbf{x}^{(k+1)} = \mathbf{x}^{(k)}$$

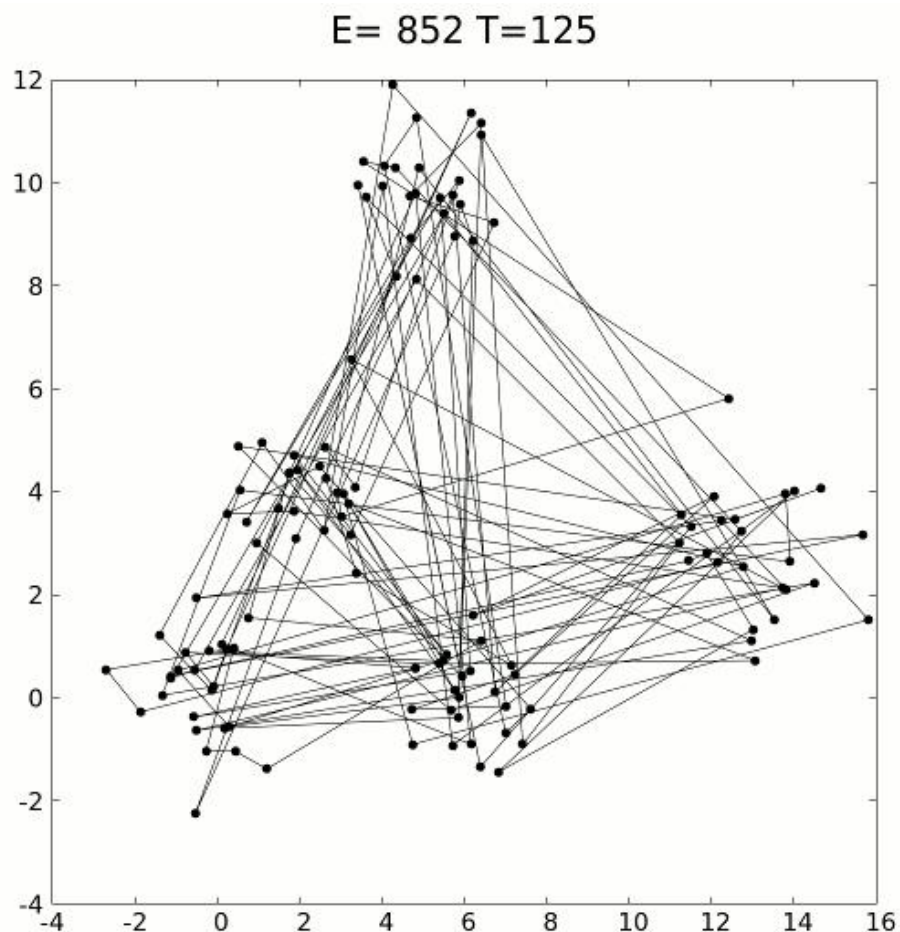
- 3.重复第1, 2步 m_j 次
- 4.增加 j 且更新 $t_j = \alpha(t_{j-1})$, $m_j = \beta(m_{j-1})$, 转至第1步
- 如果根据总迭代次数的限制或事先给定的 t_j, m_j , 算法不停止, 则可以用绝对或相对收敛准则来控制。一般停止准则由最小温度来表示, 停止后的最有候选解就是估计的最小值
- 此外, 函数 $\alpha(\cdot)$ 应使温度慢慢递减至0, 每个温度 m_j 中的迭代次数应较大且关于 j 单增, 理想的 $\beta(\cdot)$ 应使 m_j 为 p 的指数

随机下降算法: 逃脱局部最优解

7.3 随机神经网络

- 模拟退火解旅行商问题
 - 如何定义旅行商问题？

标号1-n, 任意一次旅行设为 x 表示其一个排列, 考虑如何生成 x 的一个邻域? 2领域有 $\frac{n(n-3)}{2}$ 个, 比完全解空间 $\frac{(n-1)!}{2}$ 个旅行要小很多!

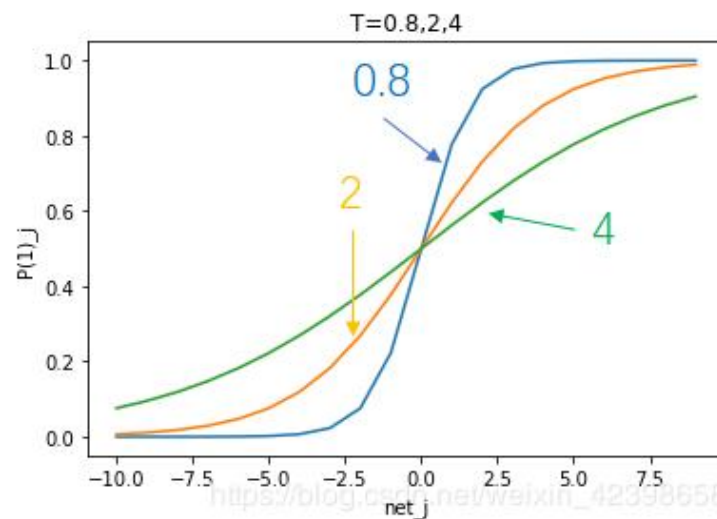
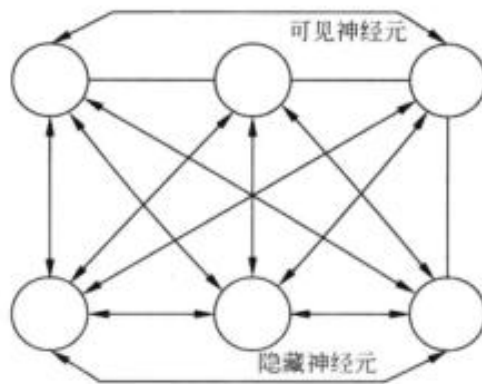


7.3随机神经网络

- 模拟
 - 定义一个网络温度以模仿物质的退火温度，取网络能量为欲优化的目标函数。网络运行开始时温度较高，调整权值时允许目标数偶尔向增大的方向变化，以使网络能跳出那些能量的局部极小点。随着网络温度不断下降至0，最终以概率1稳定在其能量数的全局最小点，从而获得最优解
- 玻尔兹曼机-随机神经网络
 - G.E.Hinton等于1983~1986，神经元只有两种输出状态，即单极性二进制的0或1。状态的取值根据概率统计法则决定，由于这种概率统计法则的表达形式与著名统计力学家Boltzmann提出的Boltzmann分布类似，故将这种网络取名BoltzmannMachine(BM).

7.3 随机神经网络

- 玻尔兹曼机
 - 结构如图所示



- 节点输入: $net_j = \sum_i^n (w_{ij}x_i - T_j)$
- 实际输出按一定概率计算, 如, 节点输出为1的概率表示为: $P_j(1) = \frac{1}{1 + e^{-\frac{net_j}{T}}}$, 输出为0的概率表示为 $1 - P_j(1)$
- 网络能量函数 E
- $E(t) = -\frac{1}{2}x^T(t)wx(t) + x^T(t)T = -\frac{1}{2}\sum_{j=1}^n \sum_{i=1}^n w_{ij}x_i x_j + \sum_{i=1}^n T_i x_i$
- BM网络按异步方式工作, 每次第j个神经元改变状态
$$\Delta E(t) = -\Delta x_j(t)net_j(t)$$
- 分析, $net_j > 0$ 和 $net_j < 0$ 都会发生什么情况?

BM网络具有从局部极小的低谷中跳出的“爬山”能力?

$$\Delta E(t) = -\Delta x_j(t) net_j(t)$$

7.3 随机神经网络

- 网络在运行过程中不断地搜索更低的能量极小值，直到达到能量的全局最小。从模拟退火法的原理可以看出，温度T不断下降可使网络能量的“爬山”能力由强减弱，这正是保证BM网络能成功搜索到能量全局最小的有效措施
- BM网络的玻尔兹曼分布
- 设 $x_j = 1, E_1$, $x_j = 0, E_0$, $x_j: 1 \rightarrow 0$, $\Delta x_j = -1$, 则 $\Delta E = E_0 - E_1 = -(-1)net_j = net_j$
 - $P_j(1) = \frac{1}{1 + e^{-\frac{net_j}{T}}} = \frac{1}{1 + e^{-\frac{\Delta E}{T}}}$, $P_j(0) = 1 - P_j(1)$
 - $\frac{P_j(1)}{P_j(0)} = \frac{e^{-\frac{E_0}{T}}}{e^{-\frac{E_1}{T}}}$, 类似的, $x_j: 0 \rightarrow 1$ 也有类似的公式, 将其推广
- 玻尔兹曼分布
 - 网络中任意两个状态出现的概率与对应能量之间的关系: $\frac{P(\alpha)}{P(\beta)} = \frac{e^{-\frac{E_\alpha}{T}}}{e^{-\frac{E_\beta}{T}}}$

7.3随机神经网络

- 玻尔兹曼分布

- BM网络处于某一状态的概率主要取决于此状态下的能量 E ，能量越低，概率越大；
- BM网络处于某一状态的概率还取决于温度参数 T ，温度越高，不同状态出现的概率越接近，网络能量较易跳出局部极小而搜索全局最小；温度低则情况相反。这正是采用模拟退火方法搜索全局最小的原因所在。

$$\frac{P(\alpha)}{P(\beta)} = \frac{e^{-\frac{E_{\alpha}}{T}}}{e^{-\frac{E_{\beta}}{T}}}$$

7.4 进化算法和遗传算法

- 借助生物学中的适者生存的规律
 - 优胜劣汰
- 遗传算法的主要特点
 - 进化发生在解的编码上，这些编码按生物学上的术语称为染色体。由于对解进行了编码，优化问题的一切性质都通过编码来研究。遗传算法涉及编码和解码
 - 自然选择规律决定哪些染色体产生超过平均数的后代。遗传算法中通过优化问题的目标人为地构造适应函数以达到好的染色体产生超过平均数的后代
 - 当染色体结合时，双亲的遗传基因的结合使得子女保持父母的特征
 - 当染色体结合后，随机的变异会造成子代同父代的不同

7.4 进化算法和遗传算法

- 用遗传算法求解 $f(x) = x^2, 0 \leq x \leq 31, x$ 为整数的最大值
 - 简单表示解的编码采用二进制表示, 采用5位二进制表达
 - 染色体: 10000:16, 11111:31, 01001:9, 00010:2
 - 每个分量称为基因, 每个基因两种状态0和1.
 - 模拟生物进化, 首先产生群体, 随机取四个染色体组成群体: $x_1 = (00000), x_2 = (11001), x_3 = (01111), x_4 = (01000)$, 群体有4个个体
 - 适应函数根据目标函数确定: $fitness(x) = f(x) = x^2$, 因此: $f(x_1) = 0, f(x_2) = 25^2, f(x_3) = 15^2, f(x_4) = 8^2$
 - 定义第 i 个个体入选种群的概率:
 - $p(x_i) = \frac{fitness(x_i)}{\sum_i fitness(x_i)}$, 显然, 适应函数值大的个体生存概率大
 - 若群体中选四个个体称为种群, 最可能选上的是 $x_2 = (11001), x_3, x_4$
 - 若他们结合, 采用如下的交叉方式, 则
 - $x_2 = (11001) \rightarrow y_1 = (11111)$
 - $x_3 = (01111) \rightarrow y_2 = (01001)$
 - $x_2 = (11001) \rightarrow y_3 = (11000)$
 - $x_4 = (01000) \rightarrow y_4 = (01001)$

7.4 进化算法和遗传算法

- 用遗传算法求解 $f(x) = x^2, 0 \leq x \leq 31, x$ 为整数的最大值
 - $x_2 = (\mathbf{11001}) \rightarrow y_1 = (11111)$ $x_2 = (\mathbf{11001}) \rightarrow y_3 = (11000)$
 - $x_3 = (\mathbf{01111}) \rightarrow y_2 = (01001)$ $x_4 = (\mathbf{01000}) \rightarrow y_4 = (01001)$
 - 若 y_4 第一个基因发生变异, $y_4 = (11001)$
- 遗传算法
 - Step1. 选择问题的一个编码; 给出一个有 N 个染色体的初始群体 $pop(1), t := 1$;
 - Step2. 对群体 $pop(t)$ 中的每个染色体 $pop_i(t)$ 计算它的适应度函数
$$f_i = fitness(pop_i(t))$$
 - Step3. 若停止准则满足, 则算法终止; 否则, 计算概率
$$p_i = \frac{f_i}{\sum_j f_j}, i = 1, 2, \dots, N$$
 - 以该概率随机在种群 $pop(t)$ 中随机选一些染色体构成一个种群
$$newpop(t+1) = \{pop_j(t) | j = 1, 2, \dots, N\}$$
 - Step4. 通过交叉, 交叉概率为 P_c , 得到一个有 N 个染色体的 $crosspop(t+1)$;
 - Step5. 以一个较小的概率 p , 使得一个染色体的一个基因发生变异, 形成 $mutpop(t+1); t := t+1$, 一个新的群体 $pop(t) = mutpop(t)$, 返回第二步 Step2.
- 模拟退火和遗传算法都可以采用马尔科夫链来做收敛分析!

7.4 进化算法和遗传算法

- 数值结果 代码参见: `genecAlgTest.ipynb`

```
=====
遗传算法求解  $f(x)=x^2$  最大值问题
=====
```

问题描述:

目标函数: $f(x) = x^2$

定义域: $0 \leq x \leq 31$, x 为整数

目标: 寻找 x 使 $f(x)$ 最大

理论最大值: $x = 31$, $f(31) = 961$

遗传算法参数:

种群大小: 50

交叉概率: 0.8

变异概率: 0.1

最大进化代数: 100

精英保留数: 2

```
=====
```

开始进化...

```
=====
Generation 0: Best x = 30, Best f(x) = 900, Avg Fitness = 288.54
Generation 10: Best x = 31, Best f(x) = 961, Avg Fitness = 653.24
Generation 20: Best x = 31, Best f(x) = 961, Avg Fitness = 695.84
Generation 30: Best x = 31, Best f(x) = 961, Avg Fitness = 684.28
Generation 40: Best x = 31, Best f(x) = 961, Avg Fitness = 697.12
Generation 50: Best x = 31, Best f(x) = 961, Avg Fitness = 685.44
Generation 60: Best x = 31, Best f(x) = 961, Avg Fitness = 740.68
Generation 70: Best x = 31, Best f(x) = 961, Avg Fitness = 756.04
Generation 80: Best x = 31, Best f(x) = 961, Avg Fitness = 731.00
Generation 90: Best x = 31, Best f(x) = 961, Avg Fitness = 722.04
Generation 99: Best x = 31, Best f(x) = 961, Avg Fitness = 717.60
=====
```

最终结果:

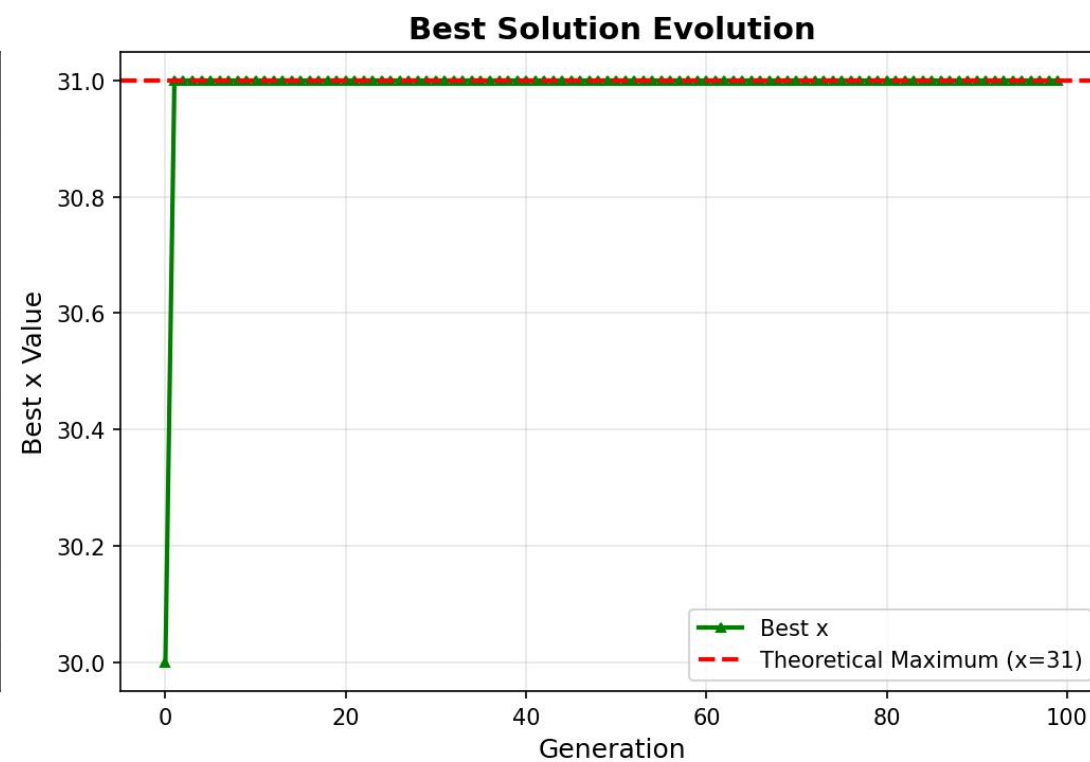
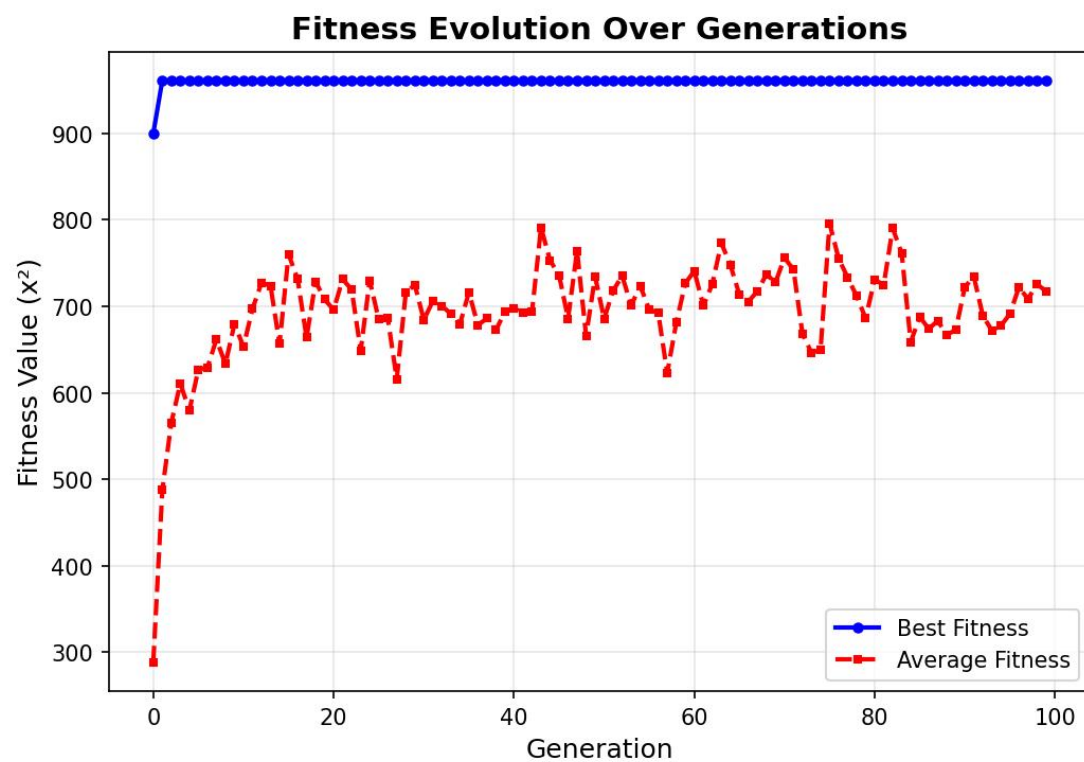
最优解: $x = 31$

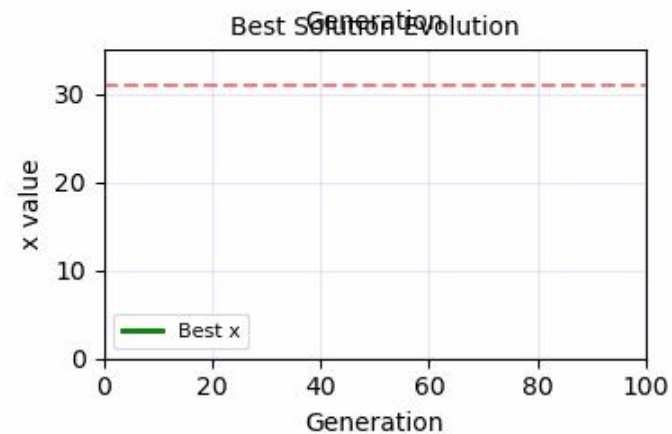
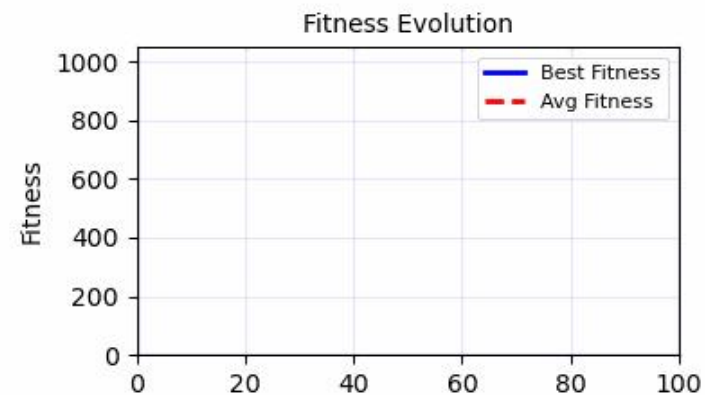
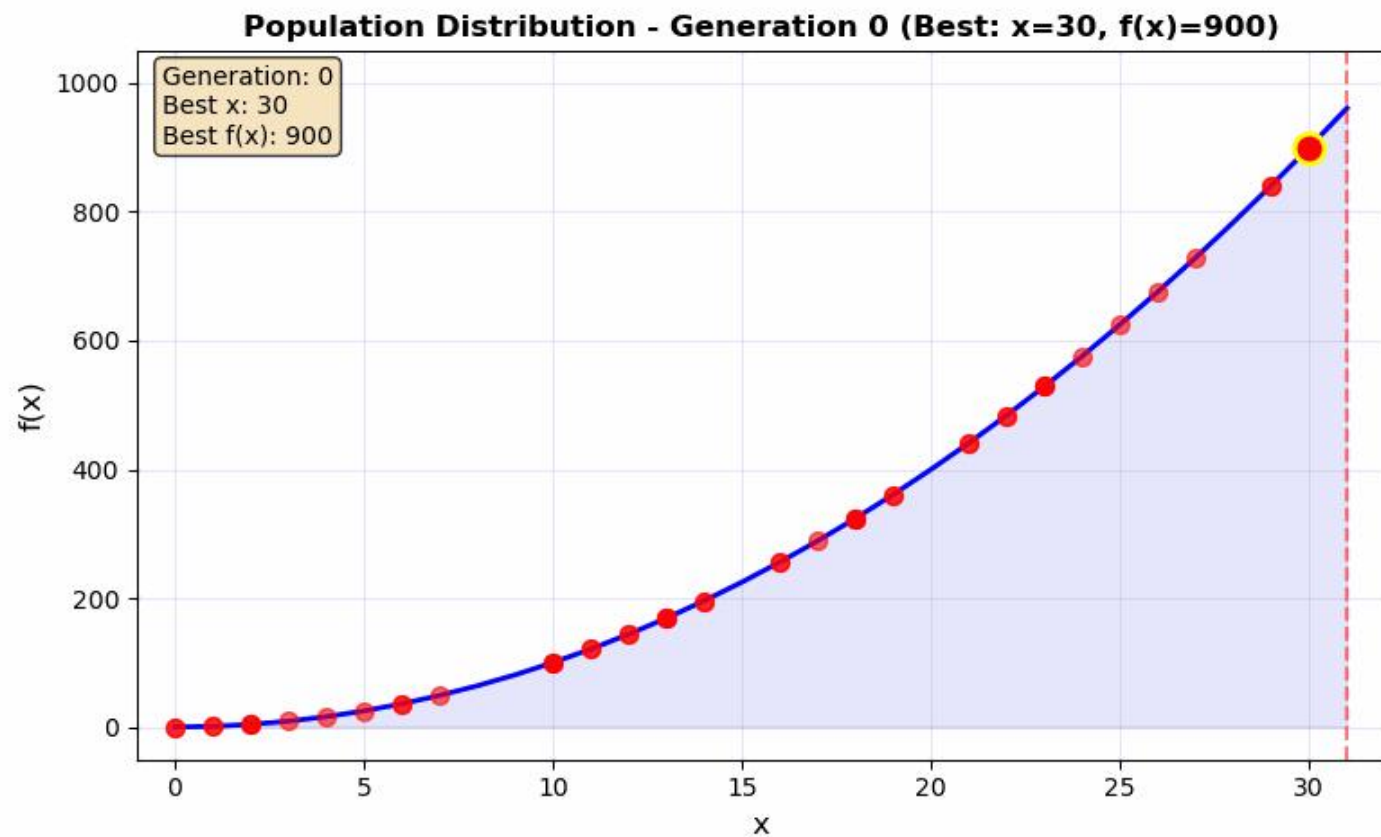
最大值: $f(x) = 961$

理论最大值: $f(31) = 961$

```
=====
```

7.4 进化算法和遗传算法





Genetic Algorithm Progress: 0.0%
Current Best: $x=30$, $f(x)=900$
Target: $x=31$, $f(x)=961$
Population Size: 50
Mutation Rate: 0.10

作业

- 用启发式算法，例如，模拟退火，玻尔兹曼机和遗传算法去求解一个非线性优化问题，掌握基本的算法思想。
- 例如： $y = (x - 2) * (x + 3) * (x + 8) * (x - 9)$ ，这种简单的非线性非凸优化问题！

遗传算法求解多项式函数极小值问题

问题描述：

目标函数： $f(x) = (x-2)(x+3)(x+8)(x-9)$

定义域： $x \in [-10, 10]$

目标：寻找 x 使 $f(x)$ 最小

函数类型：四次多项式，有多个局部极值

遗传算法参数：

编码方式：实数编码

种群大小：100

交叉概率：0.8（算术交叉）

变异概率：0.1（高斯变异）

最大进化代数：100

精英保留数：2

目标函数分析

函数形式： $f(x) = (x-2)(x+3)(x+8)(x-9)$

定义域： $x \in [-10, 10]$

找到的局部极小值点：

$x = -6.056056, f(x) = -720.574279$

$x = 6.476476, f(x) = -1549.720657$

全局最小值（数值搜索）：

$x = 6.476476$

$f(x) = -1549.720657$

边界值：

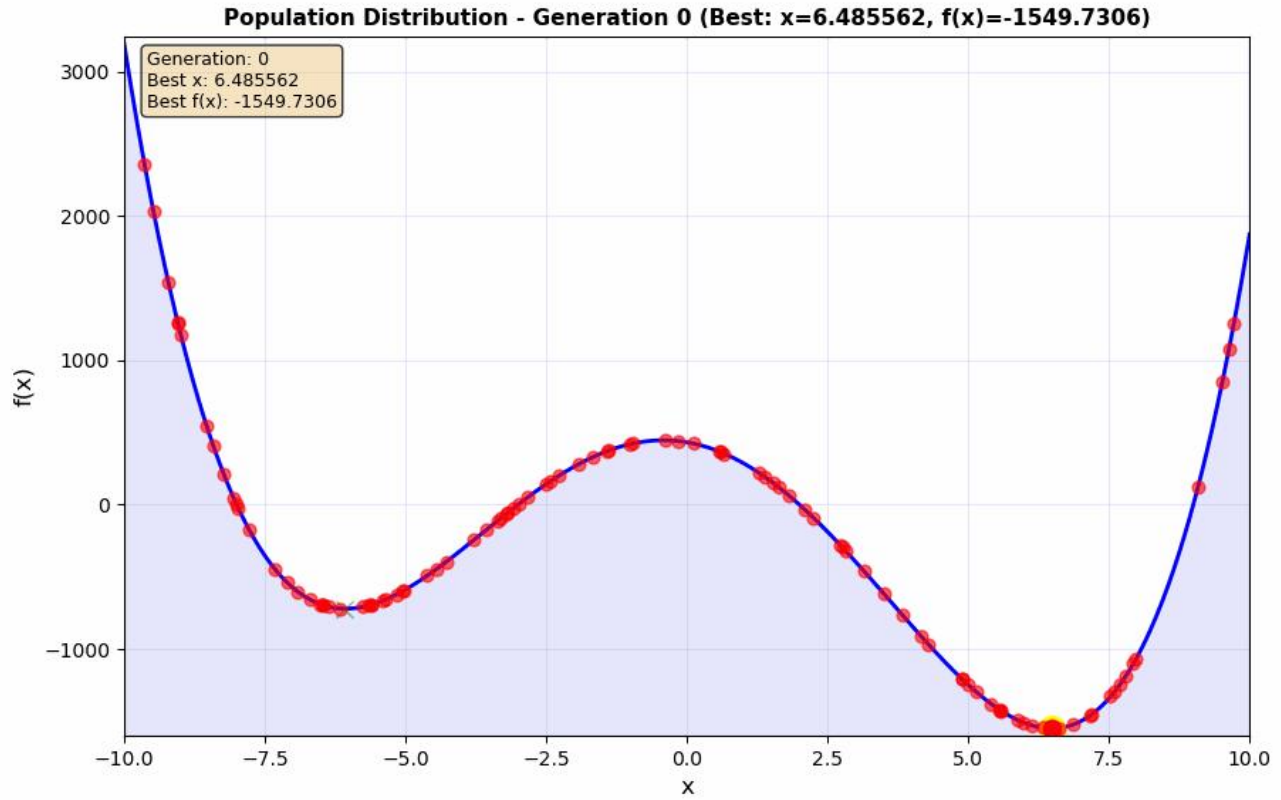
$f(-10) = 3192.000000$

$f(10) = 1872.000000$

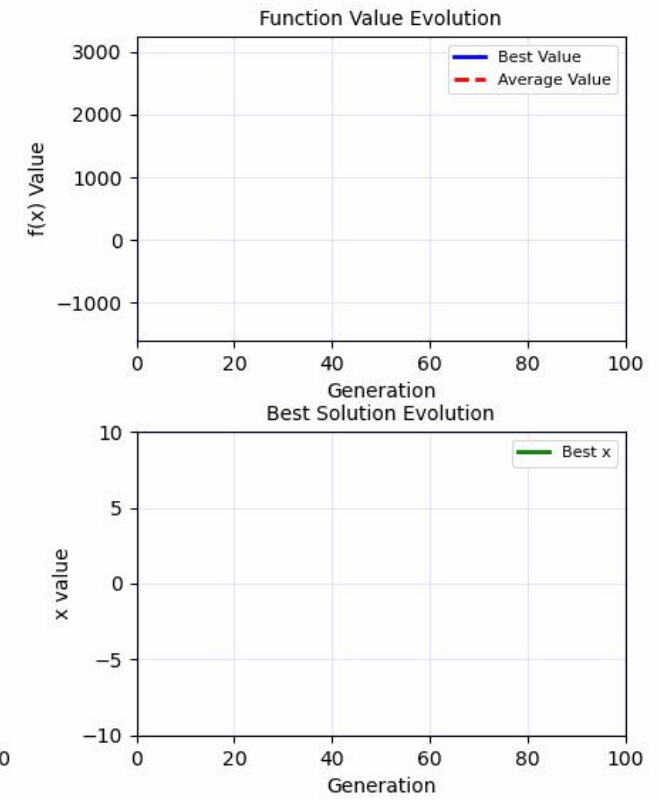
开始进化...

Generation	0:	Best x = 6.485562,	Best f(x) = -1549.7306,	Avg Value = -301.7600
Generation	10:	Best x = 6.485535,	Best f(x) = -1549.7306,	Avg Value = -1373.7021
Generation	20:	Best x = 6.485410,	Best f(x) = -1549.7307,	Avg Value = -1422.1499
Generation	30:	Best x = 6.485410,	Best f(x) = -1549.7307,	Avg Value = -1394.5579
Generation	40:	Best x = 6.483953,	Best f(x) = -1549.7309,	Avg Value = -1461.5234
Generation	50:	Best x = 6.483953,	Best f(x) = -1549.7309,	Avg Value = -1408.3390
Generation	60:	Best x = 6.483953,	Best f(x) = -1549.7309,	Avg Value = -1441.4169
Generation	70:	Best x = 6.483953,	Best f(x) = -1549.7309,	Avg Value = -1357.6002
Generation	80:	Best x = 6.483993,	Best f(x) = -1549.7309,	Avg Value = -1448.4065
Generation	90:	Best x = 6.484272,	Best f(x) = -1549.7309,	Avg Value = -1502.8069
Generation	99:	Best x = 6.484272,	Best f(x) = -1549.7309,	Avg Value = -1451.6084

Function Value Evolution (Minimization)



Best Solution Evolution



Genetic Algorithm Progress: 0.0%
Current Best: $x=6.485562$, $f(x)=-1549.7306$
Total Improvement: 0.0000
Population Size: 100
Mutation Rate: 0.10
Crossover Rate: 0.80

总结

- 除了数值优化算法还有很多别的优化方法，例如启发式算法
- 启发式算法在研究和应用中也很常见
- 各种新的启发式算法，尤其是生物现象、物理现象中抽象出来的算法。
- 机器学习方法可以用于AI for Science and Engineering，称为AI4SE土木学院李惠院士就是人工智能双聘院士，正在和计算机学院合作开展AI4S的工作，大家可以联系李老师，在综合楼7楼！

课程总结

- 介绍了向量、范数、线性空间
- 逼近定理、万能逼近定理、机器学习基本原理
- 数据噪声和分布外数据（外点数据）的一般处理思想
- 求解 $AX=b$ ，最小二乘法及其正则化，稀疏求解
- 矩阵的低秩分解：特征值分解与SVD分解
- 检测准则（贝叶斯准则、随机场）
- 信源熵与离散信源最大熵定理(极大熵原理)
- 线性规划、无约束与有约束问题的必要条件、对偶问题
- 启发式优化算法



谢 谢!

哈尔滨工业大学计算学部 刘绍辉

2026年4月

